AI & Agents — Question & Answer Reference

> 🎯 **Purpose**: Concise answers to all "Can explain" questions from the 7-day curriculum.

> 📖 **Usage**: Use this as a study guide, interview prep, or quick reference.

Day 1A: Tokens & Embeddings

Q: What is a token and why not use characters or words?

A: A token is a subword unit (e.g., "tokenization" → ["token", "ization"]). Characters are too granular for the model to learn meaning efficiently. Whole words create an impossibly large vocabulary (every misspelling, conjugation, compound word). Subwords balance vocabulary size (~100K–200K) with meaningful units.

Q: How does BPE build a vocabulary from text?

A: BPE starts with individual characters as the initial vocabulary. It then repeatedly finds the most frequent adjacent pair in the training data and merges them into a new token. After thousands of merges, common words become single tokens ("the", "and") while rare words get split into known pieces ("unforgettable" → "un" + "forget" + "table").

Q: Why are LLMs bad at arithmetic?

A: Because numbers get split into unexpected tokens. "123456" might become ["123", "456"] or ["12", "345", "6"] — the model doesn't see it as a single number. It has to learn arithmetic patterns across arbitrary token boundaries, which is unreliable. This is why tool-use (calculators) is important for agents.

Q: What is an embedding and why are token IDs insufficient?

A: Token IDs are arbitrary integers (e.g., "Hello"=9906, "Hi"=17250) — there's no way to tell they're similar. Embeddings convert each ID into a dense vector (e.g., 4096 numbers) where similar meanings land nearby in space. These vectors are learned during training so the model can do meaningful computation with them.

Q: Token embedding vs contextual embedding — what's the difference?

A: A token embedding is a static lookup — "bank" always gets the same vector regardless of context. A contextual embedding is the output AFTER the attention layers — "river bank" and "bank account" produce different vectors for "bank". The Transformer's job is to turn static token embeddings into rich contextual embeddings.

Q: What does "king − man + woman = queen" demonstrate?

A: It shows that embedding space captures semantic relationships as directions. The "gender" direction (man→woman) is consistent, so applying it to "king" lands near "queen". This means the model has learned abstract concepts like gender, tense, and category as geometric properties of the space — without being explicitly taught.

Day 1B: Transformer & Attention

Q: What does self-attention do and why is it needed?

A: Self-attention lets each token look at every other token in the sequence and decide how much to "pay attention" to each one. It's needed because word meaning depends on context — "bank" means different things near "river" vs "money". Without attention, each token would be processed in isolation with no awareness of surrounding words.

Q: Why are Q, K, V separate projections instead of comparing embeddings directly?

A: Because the question you ask (Q), the label you present to be found (K), and the content you provide (V) are different roles. A word's "searchability" is different from the information it carries. For example, "Paris" might have a Key that signals "capital city" but a Value containing facts about population and geography. Learned projections let the model separate these roles.

Q: Why do we scale by √d_k?

A: Without scaling, the dot products Q·K grow proportionally to the dimension size d_k. Large dot products push softmax into regions where gradients are extremely small (near 0 or 1), making learning impossible. Dividing by √d_k keeps the variance of the dot products at ~1, ensuring softmax produces useful gradient signal.

Q: What is multi-head attention and why use multiple heads?

A: Multi-head attention runs several attention operations in parallel, each with different learned projections. Different heads can learn different types of relationships: one head might track syntax (subject-verb), another semantics (word meaning), another coreference ("she"→"Alice"). It's like reading a sentence simultaneously for grammar, meaning, and references.

Q: What is the causal mask and why is it needed?

A: The causal mask prevents tokens from attending to future positions — token at position i can only see positions ≤ i. This is essential for autoregressive generation because the model can't use the answer to predict the answer. Implementation: set future positions to -∞ before softmax, which makes their attention weight exactly 0.

Q: What is the FFN's role vs attention's role?

A: Attention gathers relevant information from context — it's the "what should I look at?" step. The FFN (feed-forward network) processes that gathered information — it's the "what do I know about this?" step. Research shows specific FFN neurons activate for specific facts ("Paris is the capital of France" lives in particular neurons). Attention reads the right books; FFN thinks about what was read.

Q: Why do residual connections enable deep networks?

A: Without residual connections, gradients must pass through every layer during backpropagation. In deep networks (96+ layers), they shrink exponentially and vanish — early layers can't learn. Residual connections (output = layer(x) + x) create a "gradient highway" where the learning signal can flow directly through the skip connection, keeping gradients strong even at layer 1.

Day 1C: Training & Inference

Q: What is the difference between pre-training, SFT, and RLHF?

A: **Pre-training**: predict next token on trillions of tokens of internet text — learns language, facts, reasoning, but produces a text completer, not an assistant. **SFT**: fine-tune on curated (instruction, response) pairs — teaches the format of being helpful (answer directly, follow instructions). **RLHF**: human raters rank outputs, train a reward model, then optimize the LLM to maximize reward — makes it more helpful, harmless, and honest. Each phase builds on the previous.

Q: How does temperature affect generation?

A: Temperature scales the raw prediction scores (logits) before softmax. T=0: always picks the most likely token (deterministic, "boring"). T=1: samples from the model's actual learned distribution. T>1: flattens the distribution, making unlikely tokens more probable (creative but risky). T<1: sharpens the distribution, making the model more conservative. For factual tasks use low temperature; for creative tasks use higher.

Q: Why does the KV-cache exist?

A: During generation, the model predicts one token at a time and feeds it back as input. Without caching, step N would recompute attention over all N tokens from scratch — $O(N^2)$ total work. The KV-cache stores the Key and Value vectors from all previous steps, so only the new token's Query needs computation. This turns quadratic cost into linear, but requires GPU memory proportional to sequence length.

Q: Why do modern LLMs use decoder-only architecture?

A: Decoder-only is simpler (one architecture, one training objective: predict next token), scales more predictably, and produces emergent abilities at sufficient scale. A single decoder-only model can do classification, generation, translation, coding — all via prompting — without needing separate encoder/decoder stacks. Every frontier model (GPT-5, Claude Opus, LLaMA 4, Gemini 3) is decoder-only.

Day 2: Prompt Engineering

Q: What is chain-of-thought and why does it help?

A: Chain-of-thought (CoT) prompting asks the model to show its reasoning step by step before giving a final answer. It helps because LLMs generate one token at a time — without CoT, they must "jump" to the answer in one step. With CoT, intermediate tokens serve as working memory, letting the model break complex problems into manageable steps. Simply adding "Let's think step by step" can significantly improve accuracy on reasoning tasks.

Q: What are the main prompt failure modes?

A: **Instruction following failure**: the model ignores constraints (especially ones buried in the middle of long prompts). **Hallucination**: confidently generates false information. **Format breaking**: returns prose when you asked for JSON. **Sycophancy**: agrees with the user even when they're wrong. **Lost in the middle**: ignores information in the middle of long contexts. Mitigation: put critical instructions at the start AND end, use structured output formats, and add validation.

Q: Why does instruction position matter?

A: LLMs have a "primacy and recency" bias — they pay most attention to the beginning and end of the context, and can lose information in the middle (the "lost in the middle" phenomenon). Place your most critical constraints at the start (system prompt) and repeat them at the end. Never bury important instructions in the middle of a long prompt.

Q: When to use few-shot vs zero-shot?

A: Use **zero-shot** when the task is simple and well-understood by the model (sentiment analysis, basic classification) — it's cheaper and faster. Use **few-shot** when you need a specific output format, the task is unusual, or zero-shot gives inconsistent results. 2–5 examples usually suffice. More than 10 examples rarely helps — consider fine-tuning instead.

Day 3: Single Agent (ReAct)

Q: What is the ReAct pattern and why is it effective?

A: ReAct (Reason + Act) interleaves reasoning ("I need to find X"), acting (calling a tool), and observing (processing the result) in a loop. It's effective because it combines the LLM's reasoning ability with real-world tool access. The model can correct course based on observations rather than committing to a plan upfront. The explicit "Thought" step also makes the agent's reasoning transparent and debuggable.

Q: What are the three types of agent memory?

A: **Short-term (working) memory**: the current conversation context window — what the agent is actively "thinking about." **Long-term memory**: persistent storage (vector DB, files) that survives across sessions — facts, user preferences, past interactions. **Episodic memory**: records of past experiences — what worked, what failed, and why — enabling the agent to learn from its own history.

Q: What are common agent failure modes and how to prevent them?

A: **Infinite loops**: agent repeats the same action — prevent with max iteration limits and loop detection. **Tool misuse**: wrong parameters or wrong tool — prevent with clear tool descriptions and input validation. **Goal drift**: agent loses track of the original task — prevent with explicit goal tracking in the system prompt. **Hallucinated actions**: agent invents tools that don't exist — prevent with strict tool schemas and error handling.

Day 4: Planner / Executor

Q: Why is plan-then-execute better than step-by-step for complex tasks?

A: For complex multi-step tasks, step-by-step (ReAct) can lose sight of the big picture — it makes locally reasonable decisions that don't add up to a coherent solution. Plan-then-execute separates strategic thinking (Planner) from tactical execution (Executor). The Planner can use a stronger model to reason about the full problem, while the Executor uses a lighter model to carry out each step. This also enables parallelism — independent steps can execute simultaneously.

Q: What does the Critic check and why is it important?

A: The Critic validates the Executor's output against the original plan's intent — checking for correctness, completeness, and quality. Without a Critic, the system accepts the first output regardless of quality. The Critic can request re-execution, adjust the plan, or escalate to the user. It acts as a quality gate that prevents cascading errors from propagating through subsequent plan steps.

Q: What are guardrails and why are they necessary?

A: Guardrails are constraints that prevent the agent from taking dangerous or undesired actions — input validation (block prompt injection), output filtering (no harmful content), action limits (can't delete production data), cost caps (max API spend), and timeout limits. They're necessary because LLMs are probabilistic and can behave unexpectedly — guardrails provide deterministic safety boundaries around non-deterministic behavior.

Day 5: RAG + Vector Memory

Q: What is RAG and when should you use it vs fine-tuning?

A: RAG (Retrieval-Augmented Generation) retrieves relevant documents from a knowledge base and injects them into the prompt before generation. Use RAG when: data changes frequently, you need source citations, or you have large/diverse knowledge bases. Use fine-tuning when: you need a specific output style, domain-specific reasoning (medical, legal), or the knowledge is too large for context windows. RAG is cheaper, faster to set up, and updatable in real-time.

Q: How does cosine similarity work?

A: Cosine similarity measures the angle between two vectors, ignoring magnitude. cos(A,B) = (A·B)/(|A|×|B|). If two vectors point in the same direction → cos=1 (identical meaning). Perpendicular → cos=0 (unrelated). Opposite → cos=-1. It's preferred over Euclidean distance for embeddings because it focuses on the direction (meaning) rather than the length (which can vary due to token frequency).

Q: Why is hybrid search better than pure vector search?

A: Vector search is great for semantic similarity ("automobile" matches "car") but can miss exact keyword matches. Keyword search (BM25) excels at exact matches but misses semantic connections. Hybrid search combines both — using Reciprocal Rank Fusion (RRF) to merge the ranked results. This catches both semantic matches AND exact keyword matches, producing more robust retrieval.

Q: What are chunking strategies and their trade-offs?

A: **Fixed-size**: simple, predictable, but can split sentences mid-thought. **Recursive/paragraph**: splits at natural boundaries (paragraphs, sentences), preserves meaning but variable chunk sizes. **Semantic**: splits when the topic changes, best quality but most complex. Smaller chunks → more precise retrieval but less context per chunk. Larger chunks → more context but may include irrelevant content. Typical size: 256–512 tokens with 10–20% overlap.

Day 6: Multi-Agent Collaboration

Q: What are the main multi-agent orchestration patterns?

A: **Sequential pipeline**: agents run in a fixed order, each passing output to the next (simple, predictable). **Hierarchical**: a supervisor agent delegates to specialist workers (flexible, scalable). **Debate/consensus**: multiple agents propose solutions and vote or critique each other (higher quality, expensive). **Broadcast**: all agents process the same input independently, results are aggregated. Choice depends on task complexity, cost tolerance, and quality requirements.

Q: What agent roles exist and when to use each?

A: **Planner**: decomposes complex tasks into subtasks (use a strong model like GPT-5 or Claude Opus). **Executor**: carries out specific subtasks (can use lighter models). **Critic/Reviewer**: validates outputs for quality and correctness. **Researcher**: gathers information from tools or knowledge bases. **Router**: decides which specialist agent to invoke. Match the model strength to the role's difficulty — planners and critics need stronger reasoning than executors.

Q: How to prevent loops and deadlocks?

A: **Max iterations**: hard limit on how many times agents can communicate. **Timeout per step**: kill any step that runs too long. **Convergence detection**: if outputs stop changing, declare done. **Dependency graphs**: ensure no circular dependencies between agents. **Escalation**: if stuck after N attempts, escalate to a supervisor agent or the user. Always design with the assumption that agents WILL get stuck, and build escape hatches.

Q: What are the trade-offs of multi-agent vs single-agent?

A: **Multi-agent pros**: better separation of concerns, can use different models per role, handles complex tasks more robustly, enables parallelism. **Multi-agent cons**: higher cost (N× the LLM calls), more complex to debug, communication overhead, potential for miscommunication between agents. **Rule of thumb**: use a single agent for tasks one person could do in one sitting; use multi-agent when the task requires distinct expertise or would benefit from peer review.

← Back to AI & Agents Syllabus